

DevOps Tools Observability - Kibana Dashboard Setup Guide

Overview

This guide will help you set up a comprehensive Kibana dashboard for monitoring DevOps tools including Jenkins, SonarQube, Nexus IQ, Twistlock, Confluence, JIRA, OpenShift, and Ansible.

Prerequisites

- Elasticsearch 8.x or higher
- Logstash 8.x or higher
- Kibana 8.x or higher
- Sample log files generated

Step 1: Setup Elasticsearch Indices

Create index templates for each service to ensure proper field mappings:

```
# Create index template for all devops tools
PUT _index_template/devops-tools
{
  "index_patterns": ["devops-tools-*"],
  "template": {
    "settings": {
      "number_of_shards": 1,
      "number_of_replicas": 1,
      "index.lifecycle.name": "devops-logs-policy"
    },
    "mappings": {
      "properties": {
        "@timestamp": { "type": "date" },
        "timestamp": { "type": "date" },
        "service": { "type": "keyword" },
        "log_level": { "type": "keyword" },
        "severity": { "type": "keyword" },
        "tool_category": { "type": "keyword" },
        "environment": { "type": "keyword" },
        "team": { "type": "keyword" },
        "message": { "type": "text" },

        "job_name": { "type": "keyword" },
        "build_number": { "type": "long" },
        "status": { "type": "keyword" },
        "duration_ms": { "type": "long" },
        "duration_seconds": { "type": "float" },

        "project_key": { "type": "keyword" },
```



```
# Set appropriate permissions
sudo chmod 644 /var/log/devops-tools/*.json
```

Step 3: Start Logstash

```
# Start Logstash with the configuration
/usr/share/logstash/bin/logstash -f logstash-config.conf
```

Step 4: Create Index Patterns in Kibana

1. Navigate to **Stack Management > Index Patterns**
2. Create the following index patterns:
 - devops-tools-jenkins-* (Time field: @timestamp)
 - devops-tools-sonarqube-* (Time field: @timestamp)
 - devops-tools-nexus-iq-* (Time field: @timestamp)
 - devops-tools-twistlock-* (Time field: @timestamp)
 - devops-tools-confluence-* (Time field: @timestamp)
 - devops-tools-jira-* (Time field: @timestamp)
 - devops-tools-openshift-* (Time field: @timestamp)
 - devops-tools-ansible-* (Time field: @timestamp)
 - devops-tools-* (Combined pattern for all services)

Step 5: Create Visualizations

Dashboard 1: CI/CD Pipeline Health (Jenkins)

Visualizations to create:

1. **Build Success Rate (Gauge)**
 - Index pattern: devops-tools-jenkins-*
 - Metric: Percentage of SUCCESS builds
 - Formula: $(\text{count of status:SUCCESS} / \text{total count}) * 100$
2. **Build Duration Trends (Line Chart)**
 - Index pattern: devops-tools-jenkins-*
 - X-axis: @timestamp (Date Histogram)
 - Y-axis: Average duration_seconds
 - Split series: job_name
3. **Build Status Distribution (Pie Chart)**
 - Index pattern: devops-tools-jenkins-*
 - Slice by: status.keyword
 - Metric: Count
4. **Failed Builds by Job (Bar Chart)**
 - Index pattern: devops-tools-jenkins-*
 - Filter: status: FAILURE
 - X-axis: job_name.keyword

- Y-axis: Count
5. **Build Activity Timeline (Heatmap)**
 - Index pattern: devops-tools-jenkins-*
 - X-axis: @timestamp (hourly)
 - Y-axis: job_name.keyword
 - Metric: Count

Dashboard 2: Code Quality Metrics (SonarQube)

Visualizations to create:

1. **Quality Gate Pass Rate (Metric)**
 - Index pattern: devops-tools-sonarqube-*
 - Metric: Percentage of PASSED quality gates
2. **Code Coverage Trend (Area Chart)**
 - Index pattern: devops-tools-sonarqube-*
 - X-axis: @timestamp
 - Y-axis: Average coverage_percentage
 - Split series: project_key
3. **Issues by Type (Stacked Bar)**
 - Index pattern: devops-tools-sonarqube-*
 - X-axis: project_key.keyword
 - Y-axis: Sum of bugs, vulnerabilities, code_smells
 - Split series: Issue type
4. **Technical Debt Overview (Data Table)**
 - Columns: project_key, bugs, vulnerabilities, code_smells, coverage_percentage
 - Sorted by: total_issues DESC
5. **Quality Gate Status (Tag Cloud)**
 - Index pattern: devops-tools-sonarqube-*
 - Tags: quality_gate_status

Dashboard 3: Security Scanning (Nexus IQ + Twistlock)

Visualizations to create:

1. **Critical Vulnerabilities (Metric)**
 - Index pattern: devops-tools-nexus-iq-*, devops-tools-twistlock-*
 - Metric: Sum of critical_vulnerabilities + critical_cves
2. **Vulnerability Severity Distribution (Donut Chart)**
 - Index pattern: devops-tools-twistlock-*
 - Slices: critical_cves, high_cves, medium_cves, low_cves
3. **Vulnerable Images (Data Table)**
 - Index pattern: devops-tools-twistlock-*

- Columns: image_name, registry, critical_cves, high_cves, compliance_status
- Filter: critical_cves > 0 OR high_cves > 0
- 4. **Security Scan Trends (Line Chart)**
 - Index pattern: devops-tools-nexus-iq-*, devops-tools-twistlock-*
 - X-axis: @timestamp (daily)
 - Y-axis: Sum of total_vulnerabilities/total_cves
 - Split series: service
- 5. **Policy Violations by Application (Bar Chart)**
 - Index pattern: devops-tools-nexus-iq-*
 - X-axis: application_id
 - Y-axis: Sum of policy_violations

Dashboard 4: Container Platform (OpenShift/Kubernetes)

Visualizations to create:

1. **Pod Status Distribution (Pie Chart)**
 - Index pattern: devops-tools-openshift-*
 - Slice by: status.keyword
2. **Resource Usage by Namespace (Line Chart)**
 - Index pattern: devops-tools-openshift-*
 - X-axis: @timestamp
 - Y-axis: Average cpu_usage_percent, Average memory_usage_mb
 - Split series: namespace
3. **Failed Pods (Data Table)**
 - Index pattern: devops-tools-openshift-*
 - Filter: status: Failed OR status: CrashLoopBackOff
 - Columns: resource_name, namespace, restart_count, status
4. **Cluster Health Overview (Metrics)**
 - Total Pods
 - Running Pods
 - Failed Pods
 - Average CPU Usage
5. **Resource Events Timeline (Vertical Bar)**
 - Index pattern: devops-tools-openshift-*
 - X-axis: @timestamp (hourly)
 - Y-axis: Count
 - Split series: event

Dashboard 5: Automation & Configuration (Ansible)

Visualizations to create:

1. **Playbook Success Rate (Gauge)**
 - Index pattern: devops-tools-ansible-*
 - Metric: Percentage where failed: false
2. **Task Execution Times (Box Plot)**
 - Index pattern: devops-tools-ansible-*
 - Y-axis: execution_time_seconds
 - Split by: playbook
3. **Failed Tasks (Data Table)**
 - Index pattern: devops-tools-ansible-*
 - Filter: failed: true
 - Columns: playbook, task, host, message
4. **Changes by Inventory Group (Bar Chart)**
 - Index pattern: devops-tools-ansible-*
 - Filter: changed: true
 - X-axis: inventory_group
 - Y-axis: Count
5. **Automation Activity (Heatmap)**
 - X-axis: @timestamp (daily)
 - Y-axis: playbook
 - Metric: Count

Dashboard 6: Project Management (JIRA)

Visualizations to create:

1. **Issues by Type (Donut Chart)**
 - Index pattern: devops-tools-jira-*
 - Slice by: issue_type
2. **Issue Status Flow (Sankey Diagram)**
 - From: status (previous state)
 - To: status (current state)
3. **High Priority Issues (Metric)**
 - Filter: priority: Highest OR priority: High
 - Metric: Count
4. **Story Points Distribution (Histogram)**
 - X-axis: story_points
 - Y-axis: Count
5. **Work Distribution by Assignee (Bar Chart)**
 - X-axis: assignee
 - Y-axis: Count, Sum of story_points

Dashboard 7: Collaboration (Confluence)

Visualizations to create:

1. **User Activity (Line Chart)**
 - X-axis: @timestamp (hourly)
 - Y-axis: Count
 - Split series: action
2. **Most Active Spaces (Bar Chart)**
 - X-axis: space_key
 - Y-axis: Count
3. **Response Time Distribution (Histogram)**
 - X-axis: response_time_ms
 - Y-axis: Count
4. **Top Users (Data Table)**
 - Columns: user, action count
 - Sorted by: count DESC
5. **HTTP Status Codes (Pie Chart)**
 - Slice by: status_code

Step 6: Create Alerts

Navigate to **Stack Management > Rules and Connectors** to create alerts:

1. **High Build Failure Rate**
 - Trigger: When Jenkins failure rate > 30% in last hour
 - Action: Send notification
2. **Critical Vulnerabilities Detected**
 - Trigger: When critical_vulnerabilities > 0 OR critical_cves > 0
 - Action: Send urgent notification
3. **Pod Failures**
 - Trigger: When OpenShift pod status is Failed or CrashLoopBackOff
 - Action: Send notification
4. **Ansible Playbook Failures**
 - Trigger: When Ansible task fails
 - Action: Send notification
5. **Quality Gate Failure**
 - Trigger: When SonarQube quality_gate_status is FAILED
 - Action: Send notification

Step 7: Dashboard Layout

Create a main dashboard combining all visualizations:

Main Dashboard: DevOps Tools Observability

Row 1: Executive Summary - Total Build Count (Jenkins) - Build Success Rate - Critical Vulnerabilities - Failed Pods Count - Quality Gate Pass Rate

Row 2: CI/CD Health - Build Duration Trends - Build Status Distribution - Failed Builds by Job

Row 3: Code Quality - Code Coverage Trend - Issues by Type - Quality Gate Status

Row 4: Security Posture - Vulnerability Severity Distribution - Vulnerable Images - Security Scan Trends

Row 5: Platform Health - Pod Status Distribution - Resource Usage by Namespace - Cluster Health

Row 6: Automation - Playbook Success Rate - Task Execution Times - Changes by Inventory Group

Row 7: Project & Collaboration - JIRA Issues by Type - Confluence User Activity - Issue Status Flow

Step 8: Export Dashboard

Once all visualizations are created:

1. Go to **Stack Management > Saved Objects**
2. Select all dashboards and visualizations
3. Click **Export**
4. Save the export file for backup or deployment to other Kibana instances

Sample Queries for Discovery

Find all failed Jenkins builds in last 24 hours

```
service:"jenkins" AND status:"FAILURE" AND @timestamp:[now-24h TO now]
```

Find critical security vulnerabilities

```
(service:"nexus-iq" AND critical_vulnerabilities:>0) OR (service:"twistlock" AND critical_cves:>0)
```

Find failed Ansible tasks

```
service:"ansible" AND failed:true
```

Find high-priority JIRA issues

```
service:"jira" AND (priority:"Highest" OR priority:"High")
```

Find pods in failed state

```
service:"openshift" AND (status:"Failed" OR status:"CrashLoopBackOff")
```

Maintenance Tips

1. **Index Lifecycle Management (ILM)**
 - Set up ILM policy to manage index sizes
 - Roll over indices after 30 days
 - Delete old indices after 90 days
2. **Performance Optimization**
 - Use date filters in visualizations
 - Limit time range for heavy queries
 - Use aggregation sampling for large datasets
3. **Regular Updates**
 - Update index patterns when adding new fields
 - Refresh dashboards regularly
 - Review and update alerts

Troubleshooting

Logs not appearing in Kibana

- Check Logstash is running: `systemctl status logstash`
- Verify log files exist and are readable
- Check Elasticsearch indices: `GET _cat/indices/devops-tools-*`

Visualizations showing no data

- Verify index pattern is correct
- Check time range selector
- Ensure fields are mapped correctly

Performance issues

- Reduce time range
- Use filters to limit data
- Check cluster health: `GET _cluster/health`

Next Steps

1. Customize dashboards for your specific needs
2. Add more visualizations based on your use cases
3. Set up email/Slack notifications for alerts
4. Create user roles and permissions
5. Schedule PDF reports for stakeholders